

Phase 2: Requirement Analysis

AI Brand Voice & Marketing Content Generator

1. Functional Requirements (FRs)

The functional requirements delineate the system's operational boundaries, ensuring a robust, user-centric copywriting application with absolute relational integrity and fluid, interactive feedback loops.

FR ID	Function Name	Database Interacted	Functional Description
FR-1	User Accounts	USER_ACCOUNT	Registration, profile management, and secure password hashing.
FR-2	Brand Configuration	BRAND_PROFILE	Store and modify brand name, core profile metadata, and campaign purpose.
FR-3	Style Learning	SAMPLE_TEXT & BRAND_VOICE_LEARNING	Ingest raw brand text samples and trigger Gemini API to extract tone, rhythm, and vocab, saving a System Prompt Template.
FR-4	Multi-Format Gen	CONTENT_REQUEST & GENERATED_CONTENT	Generate marketing outputs across five formats using specialized few-shot prompt engineering templates.
FR-5	Feedback & Edit	CONTENT_REFINEMENT	Capture user suggestions, re-query Gemini to adjust outputs, and save the revision history relationally.

2. Non-Functional Requirements (NFRs)

- **Performance & Latency:** LLM API queries must be highly optimized. Target response latency is established at 2-3 seconds for short-form text (Taglines and Headlines) and 4-7 seconds for long-form content (Social Posts and Emails).
- **Usability & UX:** UI must provide immediate, responsive feedback. Interactive visual indicators, such as `st.spinner()` during processing and `st.success()` toast alerts on completed operations, must guide the user experience.
- **Security & Secret Management:** Strict restriction of hardcoded credentials. Enforce secure loading of Gemini API Keys from local environmental variables using `python-dotenv`.
- **Portability:** Ensure seamless replication of development environments. Enforce clean dependency management with an exact, version-locked `requirements.txt` manifest.

3. Application Technology Stack

Tech Category	Software / SDK	System Purpose	Implementation Details
AI Model	Google Gemini 2.0 Flash	Style learning and text generation.	Inference API (<code>gemini-2.0-flash</code>)
Frontend UI	Streamlit	Responsive interface, inputs, sidebars.	Python-based web app, custom CSS
Database	SQLite3	Local transactional persistence.	Relational structure, FK constraints
Helper Libs	<code>python-dotenv</code> & <code>pyperclip</code>	Secret security and UX copywriting utility.	Secure <code>.env</code> loading, click-to-copy